Name :- Rucha Nitin Pawar

Branch:- AI&DS

Div :- B

Roll No:- 70

Practical Batch :- B4

Aim :- Implementing Web Scraping in python with BeautifulSoup .

Theory

1. What is web scraping? Explain the role of the Requests library in web scraping?

Web scraping is the process of extracting data from websites automatically using programs or scripts. It allows users to collect large amounts of data from web pages for analysis or storage. The Requests library in Python plays an important role in web scraping by sending HTTP requests to websites and retrieving their HTML content. It acts as a bridge between the user and the web server, allowing the program to access web page data easily. Once the HTML content is fetched, it can be further processed using parsing libraries like BeautifulSoup.

2. What is BeautifulSoup? How does it help in parsing HTML?

BeautifulSoup is a Python library used for parsing HTML and XML documents. It helps in extracting useful information from web pages by converting complex HTML code into a structured tree format. This makes it easy to navigate, search, and modify the content. BeautifulSoup provides functions to find elements by tags, classes, or attributes, which simplifies data extraction. It is widely used in web scraping because it can handle poorly formatted HTML and makes parsing more efficient.

3. Differentiate between web scraping and web crawling?

Web scraping and web crawling are related but different concepts. Web scraping refers to extracting specific data from websites, while web crawling is the process of automatically browsing the internet and indexing web pages. Crawling is mainly used by search engines like Google to discover new web pages, whereas scraping is used to collect targeted information such as prices, reviews, or contact details. In simple terms, crawling collects links and pages, while scraping extracts useful data from those pages.

4. What are the commonly used methods in BeautifulSoup?

BeautifulSoup provides several methods that help in extracting and navigating data from HTML documents. Some commonly used methods include find(), which is used to locate the first occurrence of a tag, and find_all(), which returns all matching elements. The get_text() method is used to extract text from HTML tags, while attrs helps access attributes like class or id. The select() method allows searching using CSS selectors. These methods make it easy to locate and extract specific data from web pages.

5. What are the ethical and legal considerations in web scraping?

Web scraping must be done carefully by following ethical and legal guidelines. It is important to respect the website's terms of service and avoid scraping restricted or private data. Users should not overload servers with too many requests, as it can affect website performance. Permission should be taken if required, and sensitive or copyrighted data should not be misused. Following robots.txt rules and using scraping responsibly ensures that the process remains legal and ethical.

```
import requests
url = "https://www.geeksforgeeks.org/dsa/dsa-tutorial-learn-data-structures-and-algorithms/"
response = requests.get(url)
print(response.text)
```

```
<!DOCTYPE html><html lang="en"><head><link rel="preconnect" href="https://fonts.googleapis.com"/><link rel="preconnect" href="
    var arrPostCat = [];
    arrPostCat.push('6263');
    var arrPostCatName = "";
    var matching_category = "dsa";
    var tIds = "6263,6527,8104";
    var termsNames = "dsa,tutorials,dsatutorials";
    var tIdsInclusiveParents = "6263,6527,8104";
    var domain = 1;
    var arrPost = [];
    var post_id = "1103752";
    var post_type = "post";
    var post_slug = "dsa-tutorial-learn-data-structures-and-algorithms";
    var ip = "185.177.125.112";
    var post_title = `DSA Tutorial`;
    var post_status = "publish";
    var practiceAPIURL = "https://practiceapi.geeksforgeeks.org/";
    var practiceURL = "https://practice.geeksforgeeks.org/";
```

```
    var post_date = "2023-11-30 11:01:46";
    var commentSysUrl = "https://discuss.geeksforgeeks.org/commentEmbedV2.js";
    var link_on_code_run = '';
    var link_search_modal_top = '';
    var country_code_cf = "NL";
    var postAdApiUrlString = "6263/6527/8104/";
  </script><link rel="canonical" href="https://www.geeksforgeeks.org/dsa/dsa-tutorial-learn-data-structures-and-algorithms/"/>
      #nprogress {
        pointer-events: none;
      }
      #nprogress .bar {
        background: #29D;
        position: fixed;
        z-index: 9999;
        top: 0;
        left: 0;
        width: 100%;
        height: 3px;
      }
      #nprogress .peg {
        display: block;
        position: absolute;
        right: 0px;
        width: 100px;
        height: 100%;
        box-shadow: 0 0 10px #29D, 0 0 5px #29D;
        opacity: 1;
        -webkit-transform: rotate(3deg) translate(0px, -4px);
        -ms-transform: rotate(3deg) translate(0px, -4px);
        transform: rotate(3deg) translate(0px, -4px);
      }
      #nprogress .spinner {
        display: block;
        position: fixed;
        z-index: 1031;
        top: 15px;
        right: 15px;
      }
      #nprogress .spinner-icon {
```

```
from bs4 import BeautifulSoup
soup = BeautifulSoup(response.text, 'html.parser')
print(soup.prettify())
```

```
 </a>
 <span>
 </span>
</li>
<li value="2">
 <span>
  Data Structures :
 </span>
 <a href="https://www.geeksforgeeks.org/dsa/array-data-structure-guide/" rel="noopener" target="_blank">
  <span>
   Array
  </span>
 </a>
 <span>
  ,
 </span>
 <a href="https://www.geeksforgeeks.org/dsa/hashing-data-structure/" rel="noopener" target="_blank">
  <span>
   Hashing
```

```
import requests
from bs4 import BeautifulSoup

# URL to scrape
url = "https://example.com"

# Send HTTP request
headers = {
    "User-Agent": "Mozilla/5.0"
}
response = requests.get(url, headers=headers)

# Check if request was successful
if response.status_code == 200:
    # Parse HTML content
    soup = BeautifulSoup(response.text, "html.parser")

    # Extract page title
    title = soup.title.string
    print("Page Title:", title)

    # Extract all headings (h1 tags)
    headings = soup.find_all("h1")
    print("\nH1 Headings:")
    for h in headings:
        print(h.text.strip())

else:
    print("Failed to retrieve page:", response.status_code)
```

```
Page Title: Example Domain

H1 Headings:
Example Domain
```

Conclusion:

This practical demonstrates how to perform web scraping in Python using BeautifulSoup. It helps in extracting useful information like titles and links from web pages. Web scraping is useful for collecting data from websites for analysis, research, and automation tasks.